Hedge Fund Analysis Structure

Analysis Files

src/analytics/advanced_transaction_analysis.py

- Handles complex transaction logic including P&L attribution and cost basis calculations.
- Performs wash sale tracking, tax-efficiency reporting, and realized vs unrealized gain analysis.
- Calculates detailed turnover ratios and cash flow analysis for transaction history.

src/analytics/backtesting_engine.py

- Core engine for running historical backtests on trading strategies.
- Simulates trade execution, portfolio tracking, and performance metric calculation.
- Supports customizable initial capital, commission structures, and slippage models.

src/analytics/option_ohlc_scraper.py

- Fetches or scrapes Historical, Open, High, Low, Close (OHLC) data for option contracts.
- likely connects to external data sources to retrieve option chain history.
- preprocesses raw option data for use in options analytics.

src/analytics/options_analytics.py

- Calculates real-time Option Greeks (Delta, Gamma, Theta, Vega, Rho).
- Analyzes complex option strategies (e.g., covered calls, spreads) and visualizes payoff diagrams.
- Scans for volatility skews and potential arbitrage or high-probability setups.

src/analytics/parallel_analytics_engine.py

- Orchestrates the execution of multiple analysis modules concurrently using asyncio and ThreadPoolExecutor.
- Significantly reduces total calculation time by running independent tasks (like Risk and Sector analysis) in parallel.
- Aggregates results from various sub-analyzers into a single unified response.

src/analytics/performance_attribution.py

- Decomposes portfolio returns into alpha (active return) and beta (market return).

• Performs factor-based attribution to explain performance drivers (e.g., sector allocation vs stock selection).
• Compares portfolio performance against benchmarks like SPY or VT.

src/analytics/portfolio_optimization.py

• Implements Modern Portfolio Theory (MPT) algorithms to find optimal asset weights.
• Calculates the Efficient Frontier to maximize returns for a given risk level.
• Supports different optimization objectives like "Max Sharpe Ratio" or "Min Volatility".

src/analytics/research_development.py

• Contains advanced modules for `StrategyBacktester`, `FactorResearcher`, and `ModelValidator`.
• Uses Machine Learning (Random Forest, Linear Regression) to identify predictive factors.
• Validates models using cross-validation and out-of-sample testing.

src/analytics/return_attribution.py

• Analyzes the sources of portfolio returns over specific time periods.
• Breaks down returns into contribution from asset allocation, currency effects, and security selection.
• Provides a detailed view of how each position contributed to the overall portfolio P&L.

src/analytics/risk_analytics.py

• The primary engine for calculating portfolio risk metrics.
• Computes Value at Risk (VaR), Conditional VaR (CVaR), and maximum drawdown.
• Analyzes portfolio volatility and downside deviation.

src/analytics/risk_analytics_polars.py

• An optimized version of risk calculations using the `polars` library for high performance.
• Handles large datasets more efficiently than standard pandas implementations.
• Used for heavy computational tasks involving large transaction histories or tick data.

src/analytics/screening_engine.py

• Filters global universe of stocks based on fundamental and technical criteria.

- Allows users to define custom screens (e.g., P/E < 20, RSI < 30).
- returns lists of qualifying symbols for further analysis or trading.

src/analytics/sector_analysis.py

- Analyzes portfolio exposure across different economic sectors (Technology, Healthcare, etc.).
- Calculates sector-wise performance to identify over- or under-performing areas.
- Helps in visualizing portfolio diversification and concentration risks.

src/analytics/statistical_analysis.py

- Performs advanced statistical tests on portfolio returns.
- Calculates skewness, kurtosis, and distribution properties of returns.
- conducting correlation analysis and other statistical measures to understand return behavior.

src/analytics/strategy_backtesting.py

- A higher-level interface for backtesting specific named strategies.
- Defines rules for entry and exit logic for strategies like "Moving Average Crossover".
- Integrates with the `BacktestingEngine` to produce user-friendly reports.

src/analytics/technical_indicators.py

- Computes standard technical analysis indicators (RSI, MACD, Bollinger Bands, etc.).
- Used by both the frontend for charting and backend for strategy signals.
- likely acts as a wrapper around libraries like `pandas-ta` or `talib`.

src/analytics/trade_timing_analyzer.py

- Evaluates the execution quality of trades based on timing.
- Compares trade execution prices against daily VWAP or day range.
- Helps identify if trades are systematically executed at poor times of the day.

src/analytics/trading_operations.py

- general-purpose module for calculating trading statistics.
- Computes metrics like Win Rate, Average Profit/Loss, and Profit Factor.
- likely creates summaries of trading activity for dashboards.

src/analytics/trading_operations_analyzer.py

- A specialized or "glue" analyzer that delegates work to `AdvancedTransactionAnalyzer`.
- Provides a specific interface for "Trading Operations" related queries.
- Ensures consistent calculation of trade performance metrics.

src/analytics/transaction_processor.py

- Responsible for ingesting, cleaning, and normalizing raw transaction data.
- standardization of data formats from different brokers or file exchanges.
- Prepares data for downstream analysis by correcting dates, symbols, and numeric formats.

src/analytics/xirr_analyzer.py

- Calculates the Extended Internal Rate of Return (XIRR) for portfolios with irregular cash flows.
- essential for accurately measuring performance when there are frequent deposits and withdrawals.
- Solves for the rate of return that nets the present value of cash flows to zero.

src/monte_carlo_v3.py

- Performs Monte Carlo simulations to project future portfolio value paths.
- Uses historical volatility and drift to generate thousands of potential future scenarios.
- estimates the probability of achieving specific financial goals or hitting drawdown limits.

News Files

US_News/app_US.py

- The main Flask/backend application file for the US News module.
- Defines API routes for fetching and serving US-market specific news.
- likely orchestrates the news fetching and processing pipeline.

US_News/ta_utils.py

- specific technical analysis utilities tailored for the news application.
- likely used to generate technical signals that tag or filter news items.
- Provides lightweight technical indicators for the news feed context.

src/pulling_news_v3.py

- specific script or module for fetching news data from external APIs.
- identifying and extracting relevant financial news from raw feeds.
- May include logic for sentiment analysis or entity extraction.

Infrastructure & Connection Files

Backend Core & API

src/main_app.py

- The entry point for the main Hedge Fund Analysis application.
- Initializes the Flask app, registers blueprints (routes), and sets up middleware.
- Configures global settings like logging, database connections, and error handlers.

src/api/admin_routes.py

- Defines API endpoints specifically for administrative tasks.
- Likely handles user management, system configuration, or data maintenance.
- Protected by higher-level authentication checks.

src/api/analytics_routes.py

- The primary router for general analytics requests.
- Maps frontend requests (e.g., "get risk metrics") to specific backend analyzer functions.
- Handles parameter parsing and response formatting for analytics data.

src/api/auth_routes.py

- Manages user authentication and authorization endpoints.
- Handles login, registration, password resets, and session management.
- Integration with Supabase for user persistence.

src/api/plaid_routes_secure.py

- Secure endpoints for interacting with the Plaid API.
- Handles link token generation and exchanging public tokens for access tokens.
- Ensures sensitive banking data is handled securely during connection.

src/api/portfolio_management_routes.py

- Endpoints for managing the user's portfolio data (CRUD operations).
- Allows users to add, update, or delete portfolios and their settings.
- likely handles manual portfolio uploads.

src/api/transaction_routes.py

- API endpoints for managing and retrieving transaction history.
- Handles file uploads for transaction logs (CSV, Excel).
- Serves paginated transaction data to the frontend tables.

src/core/portfolio.py

- Defines the `Portfolio` class/data structure.
- Represents a collection of assets/positions with methods to calculate total value, weights, etc.
- The standard object used to pass portfolio state between analyzers.

src/core/transactions.py

- Defines the `Transaction` and `TransactionPortfolio` classes.
- Standardizes the representation of a single trade (symbol, date, qty, price).
- Provides helper methods for filtering and aggregating transactions.

Backend Utilities

src/utils/config.py

- Centralized configuration management for the application.
- Loads environment variables (API keys, DB URLs) and sets default values.
- Provides a consistent interface for accessing app settings.

src/utils/data_manager.py

- A centralized manager for fetching and caching market data.
- distinct from direct client calls, likely adds a caching layer or fallback logic.
- Aggregates data from multiple sources (e.g., FMP, Yahoo) for analyzers.

src/utils/plaid_handler.py

- A specialized utility class for interacting with Plaid's Python client.
- Wraps complex Plaid API calls into simpler methods for the app to use.
- Handles error handling and response parsing from Plaid.

src/utils/plaid_supabase_manager.py

- Manages the storage and retrieval of Plaid access tokens in Supabase.
- Links Plaid items (bank connections) to specific App users.
- Ensures persistent access to user bank data across sessions.

src/utils/supabase_optimizer.py

- Optimized client or wrapper for Supabase database interactions.
- likely implements batching or efficient query patterns.
- Helps reduce latency when fetching large sets of user data.

src/utils/user_secrets.py

- Manages user-specific secrets and API keys encryption/decryption.
- Ensures that sensitive user credentials are not stored in plaintext.
- Handles secure storage retrieval for third-party integrations.

Frontend Connection Scripts

web/js/app-main.js

- The main JavaScript entry point for the frontend application.
- Initializes global state, event listeners, and routing logic.
- bootstraps the UI loading process.

web/js/global-api-config.js

- Defines global API configuration constants (Base URLs, endpoints).
- Ensures consistent API addressing across different environments (dev/prod).
- likely sets up default fetch headers or interceptors.

web/js/modules/auth.js

- Frontend module for handling user authentication flows.
- Manages login forms, token storage (localStorage/cookies), and redirect logic.
- Updates UI state based on login status.

web/js/modules/plaid-connections.js

- Manages the Plaid Link UI flow and connection management.
- Handles the frontend side of linking a bank account (success/exit callbacks).
- Displays connected accounts and allows refreshing/removing them.

Hedge Fund Analysis Structure

web/js/modules/transactions.js

- Frontend logic for the Transactions view.
- Handles rendering of transaction tables, filtering, and sorting.
- Manages the upload interface for transaction files.

web/js/utils/ui-helpers.js

- collection of shared utility functions for UI manipulation.
- Includes formatters (currency, dates), notification helpers, and loading spinners.
- Reduces code duplication across different pages.